Optimasi Penataan Paket dalam Truk dengan Algoritma Pencarian Lokal

Laporan Tugas Besar 1



IF3170 Inteligensi Artifisial Tahun Akademik 2026/2027

Disusun Oleh :

Kloce Paul William Saragih - 13524040

Ariel Cornelius Sitorus - 13524085

Yayat Nurhidayat - 13223071

Insitut Teknologi Bandung

Oktober 2026

DAFTAR ISI

IF3170 Inteligensi Artifisial Tahun Akademik 2026/2027.....1

Disusun Oleh :..... 1

DAFTAR ISI.....2

1. Deskripsi Persoalan..... 4

2. Implementasi..... 5

2.1 Hill-Climbing..... 5

2.1 Simulated Annealing.....7

2.3 Genetic Algorithm..... 9

2.4 Kerangka Eksperimen dan Antarmuka.....13

2.4.1 Kerangka Eksperimen.....13

2.4.2 Antarmuka Web.....16

2.5 Data Uji.....16

2.6 Rancangan Eksperimen.....17

3. Pembahasan..... 18

3.1 Hasil dan Analisis..... 18

3.1.1. Hill climbing.....18

3.1.2 Simulated Annealing.....22

3.1.3 Genetic Algorithm..... 25

3.1.4 Perbandingan Antaralgoritma.....26

3.1.5 Pengujian skala kecil, banyak truk, dan kasus batas..... 28

4. Kesimpulan dan Saran.....29

4.1 Kesimpulan.....29

4.1 Saran..... 29

Pembagian tugas.....29

Referensi..... 30

Lampiran.....30

1. Deskripsi Persoalan

Diberikan permasalahan 3D bin packing knapsack problem. Pada permasalahan tersebut, diberikan truk dan paket dengan deskripsi sebagai berikut.

1. Truk memiliki atribut dimensi $\{w,\ l,\ h \mid w,\ l,\ h\ \in\ N\}$ dan max capacity $M \in N$
2. Paket memiliki atribut berupa:
   A. dimensi (w, l, h)
   B. Orientasi
   C. Berat
   D. Status pecah belah
   E. ETA (perkiraan waktu kedatangan paket (dalam sisa hari))

Truk dan paket memiliki ketentuan:

Permasalahan tersebut memiliki batasan:

1. Paket tidak boleh melayang. Satu paket dianggap tertopang jika berada di lantai truk atau jika terdapat paket lain yang bagian atasnya tepat berada di bawah sedikitnya satu titik alasnya.
2. Paket tidak boleh menembus dinding peti kemas, artinya seluruh bagian paket harus
3. Paket tidak boleh saling tumpang tindih.
4. Paket pecah-belah tidak boleh berada di bawah paket lain, atau dengan kata lain tidak boleh menjadi penopang. Total berat paket di dalam sebuah peti kemas tidak boleh melebihi kapasitasnya.

Diinginkan konfigurasi pengepakan paket tiga dimensi yang memenuhi batasan dan memaksimalkan fungsi objektif yaitu value paket pada truk. Pada tugas ini diimplementasikan tiga kelompok algoritma *local search*, yakni :

1. *Hill-climbing*
   a. *Steepest Ascent*
   b. *Sideways Move*
   c. *Stochastic*
   d. *Random Restart*
2. *Simulated Annealing*
3. *Genetic Algorithm*

2. Implementasi

Seluruh kode diimplementasikan dengan Python dan *Matplotlib* untuk visualisasi. Detail dari tiap file program adalah sebagai berikut:

File	Isi
core/model.py	Struktur data (Orientation, Dimensions, Package, Truck, Placement, State, Result)
core/constraints.py	Pengecekan kendala (violations, is_valid) dan fungsi settle
core/objective.py	Fungsi objektif total_value
core/moves.py	Langkah neighbor (relocate, swap, rotate, transfer)
core/generator.py	Pembuatan state awal acak
core/io.py, core/stats.py	*Read/Write* Instance dan state (JSON), statistik per truk
algorithms/	hill_climbing.py, simulated_annealing.py, genetic_algorithm.py
run.py, run_common.py	Program utama dan harness eksperimen (tiga run, simpan hasil dan plot)
visualize.py	Visualisasi 3D peti kemas dan grafik per truk
generate_instance.py	Pembuat instance acak yang bisa diulang dengan seed
tests/	Unit test

2.1 Hill-Climbing

Keempat varian hill climbing berbagi satu fungsi inti, climb(), sehingga perbedaan perilaku murni berasal dari aturan pemilihan dan penghentian. Pada setiap iterasi diambil n_neighbors tetangga valid, dihitung value-nya, lalu dipilih satu tetangga sebagai langkah berikutnya.

Varian	Aturan Langkah	Perhentian
Steepest ascent	Pilih tetangga dengan value tertinggi dari sampel.	Tidak ada tetangga yang lebih baik.
Sideways move	Seperti steepest, tetapi boleh berpindah ke tetangga bernilai sama (dataran) hingga max_sideways kali berturut-turut.	Sideways habis atau tetangga lebih buruk semua.
Stochastic	Pilih acak satu dari tetangga yang lebih baik; bila tidak ada, ambil yang terbaik.	Tidak ada yang lebih baik.
Random restart	Pilih acak satu dari tetangga yang lebih baik; bila tidak ada, ambil yang terbaik.	Seluruh restart selesai.

Karena tetangga hanya disampel (bukan seluruh ruang tetangga), "steepest" berarti terbaik di antara sampel. Sideways dengan max_sideways = 0 secara matematis identik dengan steepest ascent, dan random restart dengan max_restarts = 0 identik dengan satu kali steepest. Kedua kesetaraan ini kami pakai sebagai uji kebenaran implementasi (hasilnya sama persis pada seed yang sama).

```
def climb(inst, rng, state, mode, max_iter, n_neighbors, max_sideways):
    cur = state
    cur_val = total_value(cur, inst)
    history = [cur_val]
```

```
    sideways = 0

    for _ in range(max_iter):
        neighbors = sample_neighbors(cur, inst, rng, n_neighbors)
        if len(neighbors) == 0:
            break
        scored = [(total_value(nb, inst), nb) for nb in neighbors]

        if mode == "stochastic":
            better = [s for s in scored if s[0] > cur_val]
            best_val, best = rng.choice(better) if better else max(scored, key=lambda s: s[0])
        else:
            best_val, best = max(scored, key=lambda s: s[0])

        if best_val > cur_val:
            sideways = 0
        elif best_val == cur_val and sideways < max_sideways:
            sideways += 1
        else:
            break

        cur, cur_val = best, best_val
        history.append(cur_val)

    return cur, history, {"sideways_used": sideways}

def single_run(inst, seed, init_state, mode, max_iter, n_neighbors, max_sideways):
    rng = random.Random(seed)
    start = time.time()
    initial = init_state if init_state is not None else random_state(inst, rng)
    final, history, extras = climb(inst, rng, initial, mode, max_iter, n_neighbors, max_sideways)
    return Result(initial, final, total_value(final, inst), history, time.time() - start, extras)
```

```
# 1. Steepest ascent: ambil neighbor terbaik, berhenti jika tidak ada yang lebih baik.
def steepest_ascent(inst, seed=None, init_state=None, max_iter=1000, n_neighbors=50):
    return single_run(inst, seed, init_state, "steepest", max_iter, n_neighbors, 0)

# 2. Sideways move: sama seperti steepest, tapi boleh pindah ke neighbor bernilai sama.
def sideways_move(inst, seed=None, init_state=None, max_iter=1000, n_neighbors=50,
max_sideways=100):
    return single_run(inst, seed, init_state, "steepest", max_iter, n_neighbors, max_sideways)

# 3. Stochastic: pilih acak satu dari neighbor yang lebih baik.
def stochastic(inst, seed=None, init_state=None, max_iter=1000, n_neighbors=50):
    return single_run(inst, seed, init_state, "stochastic", max_iter, n_neighbors, 0)

# 4. Random restart: ulangi steepest ascent dari state acak, ambil hasil terbaik.
def random_restart(inst, seed=None, init_state=None, max_iter=1000, n_neighbors=50,
max_restarts=10):
    rng = random.Random(seed)
    start = time.time()
    initial = init_state if init_state is not None else random_state(inst, rng)

    final, best_val = initial, -1
    history, per_restart, iters_per_restart = [], [], []
    for r in range(max_restarts + 1):
        s0 = initial if r == 0 else random_state(inst, rng)
        s, h, _ = climb(inst, rng, s0, "steepest", max_iter, n_neighbors, 0)
        history += h
        per_restart.append(h[-1])
        iters_per_restart.append(len(h) - 1)
        if h[-1] > best_val:
            final, best_val = s, h[-1]

    extras = {"restarts": max_restarts, "iterations_per_restart": iters_per_restart,
              "value_per_restart": per_restart}
```

```
return Result(initial, final, total_value(final, inst), history, time.time() - start, extras)
```

2.1 Simulated Annealing

Implementasi mengikuti standar yang digunakan pada slide perkuliahan IF3170. Pada setiap temperatur dilakukan *iterations_per_temp* percobaan. *Neighbor* yang lebih baik atau saam selalu diterima, sedangkan yang lebih buruk diterima dengan peluang $e^{\Delta E/T}$. T emperatur diturunkan secara geometrik. State terbaik selama pencarian akan disimpan dan dikembalikan menjadi hasil akhir. Dengan nilai bawaan (T awal 100, T iminimum 0.01, laju 0.95, 50 percobaan per temperator) terdapat 180 tingkat temperatur atau 9000 percobaan. Nilai paket pada instance berada di antara 10 dan 100, sehingga T awal 100 membuat sebagian besar langkah turun masih diterima di awal pencarian.

```
import math
import time

from core.model import Result
from core.moves import random_neighbor
from core.objective import total_value



def simulated_annealing(inst, initial_state, rng, initial_temp=100.0,
                        min_temp=0.01, cooling_rate=0.95,
                        iterations_per_temp=50):
    start = time.perf_counter()

    current = initial_state
    current_value = total_value(current, inst)
    best = current
    best_value = current_value

    history = [current_value]
```

```
prob_history = []
stuck_count = 0

temp = initial_temp
while temp > min_temp:
    for _ in range(iterations_per_temp):
        neighbor = random_neighbor(current, inst, rng)
        if neighbor is None:
            continue

        neighbor_value = total_value(neighbor, inst)
        delta = neighbor_value - current_value

        if delta >= 0:
            prob = 1.0
        else:
            prob = math.exp(delta / temp)
        prob_history.append(prob)

        if delta >= 0 or rng.random() < prob:
            current = neighbor
            current_value = neighbor_value

        if neighbor_value > best_value:
            best = neighbor
            best_value = neighbor_value
        else:
            stuck_count += 1

        history.append(current_value)

    temp *= cooling_rate
```

```
    duration = time.perf_counter() - start
    extras = {
        "stuck_count": stuck_count,
        "final_temperature": temp,
        "prob_history": prob_history,
    }

    return Result(initial_state, best, best_value, history, duration, extras)
```

2.3 Genetic Algorithm

State yang ditentukan adalah satu individu dan satu gen merupakan placement paket pada truk. Pada algoritma ini, fitness function yang dipakai sama dengan fungsi objektif pada permasalahan, yaitu nilai value paket yang ditempatkan di truk. Pada satu generasi memiliki alur seleksi, crossover, dan mutasi mengikuti salindia. Parameter yang divariasikan adalah ukuran populasi dan iterasi. Nilai maksimum dan rata-rata populasi dicatat pada setiap generasi untuk kebutuhan eksperimen.

Pada proses seleksi, pemilihan orang tua menggunakan roulette wheel dengan jumlah kumulatif dan pencarian biner, bila seluruh fitness nol orang tua dipilih secara random. Lalu crossover memotong gen pada satu titik di urutan paket. Apabila crossover melanggar constraint, maka fungsi repair menempatkan ulang paket per truk dengan mempertimbangkan nilai z lama dan nilai tertinggi lebih dulu. Paket yang melanggar batas, kapasitas, dan pecah-belah dikeluarkan. Setelah itu, paket yang berada di luar dicoba ditempatkan lagi dengan posisi acak dengan 10 percobaan per paket. Urutan percoban tesebut menggunakan heuristik rasio nilai paket terhadap sumber daya yang dipakai (value/(berat/kapasitas + volume paket/volume truk)), rasio yang besar dicoba lebih dulu. Selain itu, mutasi ditentukan dengan peluang 0.2 dengan mutasi anak memakai satu neighbor acak dari fungsi random neighbor. Dalam pergantian generasinya, seluruh populasi diganti oleh anak.

```
import random
import time

from core.constraints import in_bounds, is_valid, xy_overlap
from core.generator import random_state
```

```
from core.model import Orientation, Placement, Result, State
from core.moves import random_neighbor
from core.objective import total_value

MUTATION_PROB = 0.2
REFILL_TRIES = 10



def fitness(state, inst):
    return total_value(state, inst)

def random_selection(population, cumulative, rng):
    total = cumulative[-1]
    if total <= 0:
        return rng.choice(population)
    r = rng.uniform(0, total)
    lo, hi = 0, len(cumulative)
    while lo < hi:
        mid = (lo + hi) // 2
        if cumulative[mid] < r:
            lo = mid + 1
        else:
            hi = mid
    idx = lo
    return population[min(idx, len(population) - 1)]



def try_place(done, weight, pkg, truck, x, y, orient, inst):
    d = pkg.dims.oriented(orient)
    if x < 0 or y < 0 or x + d.w > truck.dims.w or y + d.l > truck.dims.l:
        return None
    if weight + pkg.weight > truck.max_capacity:
        return None
```

```
    footprint = (x, y, 0, d.w, d.l, d.h)
    z = 0
    for _, b in done:
        if xy_overlap(footprint, b):
            z = max(z, b[2] + b[5])
    box = (x, y, z, d.w, d.l, d.h)
    if not in_bounds(box, truck):
        return None
    if z > 0:
        for other, b in done:
            if inst.by_id[other].is_fragile and b[2] + b[5] == z and xy_overlap(box, b):
                return None
    return box



def repair(state, inst, rng=None, refill_tries=0, refill_order=()):
    new_placements = {pid: None for pid in state.placements}
    per_truck = {}
    for pid, pl in state.placements.items():
        if pl is not None and 0 <= pl.truck < len(inst.trucks):
            per_truck.setdefault(pl.truck, []).append(pid)

    done = {t: [] for t in range(len(inst.trucks))}
    weight = {t: 0 for t in range(len(inst.trucks))}

    for truck_id, ids in per_truck.items():
        truck = inst.trucks[truck_id]
        ids.sort(key=lambda p: (state.placements[p].z, -inst.by_id[p].value, p))
        for pid in ids:
            pkg = inst.by_id[pid]
            pl = state.placements[pid]
            box = try_place(done[truck_id], weight[truck_id], pkg, truck, pl.x, pl.y, pl.orient, inst)
            if box is None:
```

```
            continue
        new_placements[pid] = Placement(truck_id, box[0], box[1], box[2], pl.orient)
        done[truck_id].append((pid, box))
        weight[truck_id] += pkg.weight

  if rng is not None and refill_tries > 0:
    orients = list(Orientation)
    for pid in refill_order:
      if new_placements[pid] is not None:
        continue
      pkg = inst.by_id[pid]
      for _ in range(refill_tries):
        truck_id = rng.randrange(len(inst.trucks))
        truck = inst.trucks[truck_id]
        orient = rng.choice(orients)
        d = pkg.dims.oriented(orient)
        if d.w > truck.dims.w or d.l > truck.dims.l:
          continue
        x = rng.randint(0, truck.dims.w - d.w)
        y = rng.randint(0, truck.dims.l - d.l)
        box = try_place(done[truck_id], weight[truck_id], pkg, truck, x, y, orient, inst)
        if box is not None:
          new_placements[pid] = Placement(truck_id, box[0], box[1], box[2], orient)
          done[truck_id].append((pid, box))
          weight[truck_id] += pkg.weight
          break
  return State(new_placements)


def reproduce(x, y, inst, rng, refill_tries=0, refill_order=()):
  ids = [p.id for p in inst.packages]
  cut = rng.randint(1, max(1, len(ids) - 1))   # 1 paket: seluruhnya dari x
  placements = {}
```

```
    for i, pid in enumerate(ids):
        parent = x if i < cut else y
        placements[pid] = parent.placements[pid]
    return repair(State(placements), inst, rng, refill_tries, refill_order)



def mutate(state, inst, rng):
    neighbour = random_neighbor(state, inst, rng)
    return neighbour if neighbour is not None else state



def refill_order_of(inst):
    t = inst.trucks[0]
    vol = t.dims.w * t.dims.l * t.dims.h

    def score(p):
        return p.value / (p.weight / t.max_capacity + p.dims.w * p.dims.l * p.dims.h / vol + 1e-9)

    return [p.id for p in sorted(inst.packages, key=score, reverse=True)]

# sama kayak yang di PPT random crossover, crossover, lalu mutation
# tapi gene-nya itu penempatan satu paket dengan constraints-nya
def genetic_algorithm(inst, population_size, iterations, seed=None,
                      mutation_prob=MUTATION_PROB, refill_tries=REFILL_TRIES):
    rng = random.Random(seed)
    order = refill_order_of(inst)
    start = time.perf_counter()

    population = [random_state(inst, rng) for _ in range(population_size)]
    fits = [fitness(s, inst) for s in population]

    best_idx = max(range(population_size), key=lambda i: fits[i])
    initial_state = population[best_idx]
```

```
history_max = [max(fits)]
history_avg = [sum(fits) / len(fits)]

for _ in range(iterations):
    cumulative = []
    current_sum = 0
    for f in fits:
        current_sum += f
        cumulative.append(current_sum)

    new_population = []
    for _ in range(population_size):
        x = random_selection(population, cumulative, rng)
        y = random_selection(population, cumulative, rng)
        child = reproduce(x, y, inst, rng, refill_tries, order)
        if rng.random() < mutation_prob:
            child = mutate(child, inst, rng)
        new_population.append(child)
    population = new_population
    fits = [fitness(s, inst) for s in population]
    history_max.append(max(fits))
    history_avg.append(sum(fits) / len(fits))

best_idx = max(range(population_size), key=lambda i: fits[i])
final_state = population[best_idx]
assert is_valid(final_state, inst)

return Result(
    initial_state=initial_state,
    final_state=final_state,
    final_value=fits[best_idx],
    history=history_max,
```

```
        duration=time.perf_counter() - start,
        extras={
            "population_size": population_size,
            "iterations": iterations,
            "mutation_prob": mutation_prob,
            "history_avg": history_avg,
        },
    )
```

2.4 Kerangka Eksperimen dan Antarmuka

2.4.1 Kerangka Eksperimen

Kerangka eksperimen ditulis manual. Kelas dasar Experiment menjalankan satu konfigurasi sebanyak runs kali dengan seed berurutan (0, 1, 2), memvalidasi setiap hasil akhir, lalu menyimpannya. Fungsi sweep() membuat daftar eksperimen dengan memvariasikan satu parameter sementara parameter lain dipegang tetap, sehingga pengaruh satu parameter dapat dipisahkan.

experiments/experiment.py

```
class Experiment:
    name = "experiment"
    family = "other"          # folder hasil: hill_climbing, simulated_annealing, genetic_algorithm
    instance_name = ""        # nama data yang dipakai, diisi run_experiment

    def __init__(self, inst, runs=3, seed=0, **params):
        self.inst = inst
        self.runs = runs
        self.seed = seed
        self.params = params
        self.records = []
```

```
def run_once(self, seed):
    raise NotImplementedError

def run(self):
    self.records = []
    for i in range(self.runs):
        result = self.run_once(self.seed + i)
        assert is_valid(result.final_state, self.inst), "final state tidak memenuhi constraint"
        self.records.append(self.make_record(i, result))
    return self.records

def make_record(self, i, result):
    record = {
        "run": i,
        "seed": self.seed + i,
        "initial_state": state_to_dict(result.initial_state),
        "final_state": state_to_dict(result.final_state),
        "initial_value": total_value(result.initial_state, self.inst),
        "final_value": result.final_value,
        "iterations": len(result.history) - 1,
        "duration": result.duration,
        "history": result.history,
    }
    record.update(result.extras)
    return record

def summary(self):
    values = []
    durations = []
    iterations = []
    for record in self.records:
        values.append(record["final_value"])
        durations.append(record["duration"])
```

```
            iterations.append(record["iterations"])

        return {
            "name": self.name,
            "instance": self.instance_name,
            "params": self.params,
            "mean": statistics.mean(values),
            "best": max(values),
            "stdev": statistics.pstdev(values),
            "mean_duration": statistics.mean(durations),
            "mean_iterations": statistics.mean(iterations),
        }

    def save(self, folder="output"):
        os.makedirs(folder, exist_ok=True)

        tag = ""
        for key in sorted(self.params):
            tag += "_" + key + str(self.params[key])

        path = os.path.join(folder, self.instance_name + "_" + self.name + tag + ".json")
        with open(path, "w") as f:
            f.write(to_json({"summary": self.summary(), "records": self.records}))
        return path

def sweep(experiment_class, inst, param, values, runs=3, **fixed):
    experiments = []
    for value in values:
        params = dict(fixed)
        params[param] = value
        experiments.append(experiment_class(inst, runs=runs, **params))
    return experiments
```

experiments/hc_experiment.py (snippet)

```
class HCExperiment(Experiment):
    """Dasar semua eksperimen hill climbing: subclass cukup mengisi algorithm dan nilai sweep."""
    family = "hill_climbing"
    algorithm = None

    def run_once(self, seed):
        return self.algorithm(self.inst, seed=seed, **self.params)

class SidewaysExperiment(HCExperiment):
    name = "sideways_move"
    algorithm = staticmethod(sideways_move)

    SIDEWAYS_VALUES = [0, 10, 50]      # 0 = sama dengan steepest ascent
    NEIGHBOR_CONTROL = 30
    ITER_CONTROL = 300

    @classmethod
    def vary_sideways(cls, inst, runs=3):
        return sweep(cls, inst, "max_sideways", cls.SIDEWAYS_VALUES, runs,
                     n_neighbors=cls.NEIGHBOR_CONTROL, max_iter=cls.ITER_CONTROL)

def all_hill_climbing(inst, runs=3):
    """Semua sweep hill climbing: 4 algoritma."""
    return (SteepestExperiment.vary_neighbors(inst, runs)
          + StochasticExperiment.vary_neighbors(inst, runs)
          + SidewaysExperiment.vary_sideways(inst, runs)
          + RandomRestartExperiment.vary_restarts(inst, runs))
```

experiments/run_experiment.py

```
def build(group, inst, runs):
    experiments = []
    if group == "base":      # satu konfigurasi default tiap algoritma, untuk menguji banyak kasus
```

```
sekaligus
        return base(inst, runs)
    if group in ("ga", "all"):
        experiments += GAExperiment.vary_iterations(inst, runs) + GAExperiment.vary_population(inst,
runs)
    if group in ("sa", "all"):
        experiments += SAExperiment.vary_cooling(inst, runs)
    if group in ("hc", "all"):
        experiments += all_hill_climbing(inst, runs)
    return experiments

def base(inst, runs):
    return ([GAExperiment(inst, runs=runs, iterations=GAExperiment.ITERATION_CONTROL,
                          population_size=GAExperiment.POPULATION_CONTROL),
             SAExperiment(inst, runs=runs, initial_temp=50, min_temp=1, cooling_rate=0.8,
iterations_per_temp=10)]
            + [cls(inst, runs=runs, n_neighbors=30, **extra) for cls, extra in (
                (SteepestExperiment, {"max_iter": 300}), (StochasticExperiment, {"max_iter": 300}),
                (SidewaysExperiment, {"max_iter": 300, "max_sideways": 50}),
                (RandomRestartExperiment, {"max_iter": 200, "max_restarts": 5}))])
```

File hasil disimpan di output/experiments/<keluarga>/<data>_<nama>_<parameter>.json dan berisi ringkasan (rata-rata, terbaik, simpangan baku, rata-rata waktu dan iterasi) serta seluruh rekaman per run (state awal dan akhir, riwayat nilai, dan metrik khusus seperti sideways_used, stuck_count, atau history_avg).

2.4.2 Antarmuka Web

Server Python menyediakan API:

- /api/instances
- /api/algorithms
- /api/experiments
- /api/instance/<nama>
- /api/run

- /api/job/<id>

Proses algoritma berjalan di thread latar dan antarmuka melakukan polling. Antarmuka menampilkan peta 3D isometrik (Three.js) dengan label paket dan pelindung label yang tertutup paket lain, kartu detail paket, grafik (Recharts) untuk nilai objektif, peluang penerimaan SA, iterasi stuck, nilai per truk, dan populasi GA, serta panel eksperimen yang membaca hasil di output/ lalu menjalankan ulang konfigurasi yang dipilih.

2.5 Data Uji

Lima data uji dipilih agar setiap kasus penting teruj. Data dasar menguji perilaku normal pada skala kecil, skala menengah, dan banyak truk berukuran berbeda. Dua data edge menguji kasus batas.

Nama data	Jenis	Truk	Paket	Truk (w x l x h / kap.)	Total Value	Total Bobot	Fragile
dasar_1truk_8paket	Dasar, 1 truk	1	8	6x8x4 / 120	465	120	2
dasar_1truk_50paket	Dasar, 1 truk	1	50	10x20x5 / 500	2906	775	13
dasar_3truk_beda_ukuran_50paket	Dasar, 3 truk	3	50	10x20x5 / 500; 8x16x6 / 400; 6x12x4 / 250	2906	775	13
edge_1paket	Edge case	1	1	4x4x4 / 50	30	10	0
edge_semua_kelebihan_berat	Edge case	1	3	4x4x4 / 50	150	191	1

Data uji yang dipilih memiliki ketentuan sebagai berikut:

- dasar_1truk_8paket: bobot total 120 persis sama dengan kapasitas, jadi seluruh 8 paket (value 465) dapat terangkut dan nilai optimum diketahui.
- dasar_1truk_50paket: kasus utama, tempat semua sapuan parameter dijalankan.
- dasar_3truk_beda_ukuran_50paket: tiga truk berukuran dan kapasitas berbeda, menguji operator transfer dan pemilihan truk.

- edge_1paket: hanya satu paket, menguji batas ukuran populasi dan operator swap.
- edge_semua_kelebihan_berat: setiap paket lebih berat daripada kapasitas truk sehingga tidak satu pun dapat dimuat (optimum 0), menguji program yang tidak boleh gagal ketika tidak ada solusi bernilai.

2.6 Rancangan Eksperimen

Setiap konfigurasi dijalankan 3 kali dengan seed 0, 1, dan 2. Seed yang sama menghasilkan state awal yang sama pada konfigurasi berbeda (kecuali GA yang memakai populasi), sehingga perbandingan berpasangan wajar. Metrik: nilai akhir (rata-rata, terbaik, simpangan baku populasi), jumlah iterasi, dan durasi dinding per run.

Algoritma	Variabel Search	Nilai (parameter lain tetap)
Steepest ascent	n_neighbors	10, 30, 100 (max_iter 300)
Stochastic	n_neighbors	10, 30, 100 (max_iter 300)
Sideways move	max_sideways	0, 10, 50 (n_neighbors 30, max_iter 300)
Random restart	max_restarts	0, 5, 20 (n_neighbors 30, max_iter 200)
Simulated annealing	cooling_rate	0,5; 0,8; 0,95 (T0 50, Tmin 1, 10 iterasi per suhu)
Genetic algorithm	population_size; iterations	populasi 5, 20, 30, 100 (50 generasi); generasi 10, 50, 150 (populasi 20)
Semua, pada 4 data lain	konfigurasi dasar	satu konfigurasi per algoritma (6 per data, 24 berkas)

Total terdapat 45 berkas hasil (21 + 4 x 6). Percobaan dilakukan di mesin virtual bersama sehingga durasi absolut hanya cocok untuk perbandingan relatif.

3. Pembahasan

3.1 Hasil dan Analisis

3.1.1. Hill climbing

Berikut adalah hasil climbing pada dasar_1truk_50paket

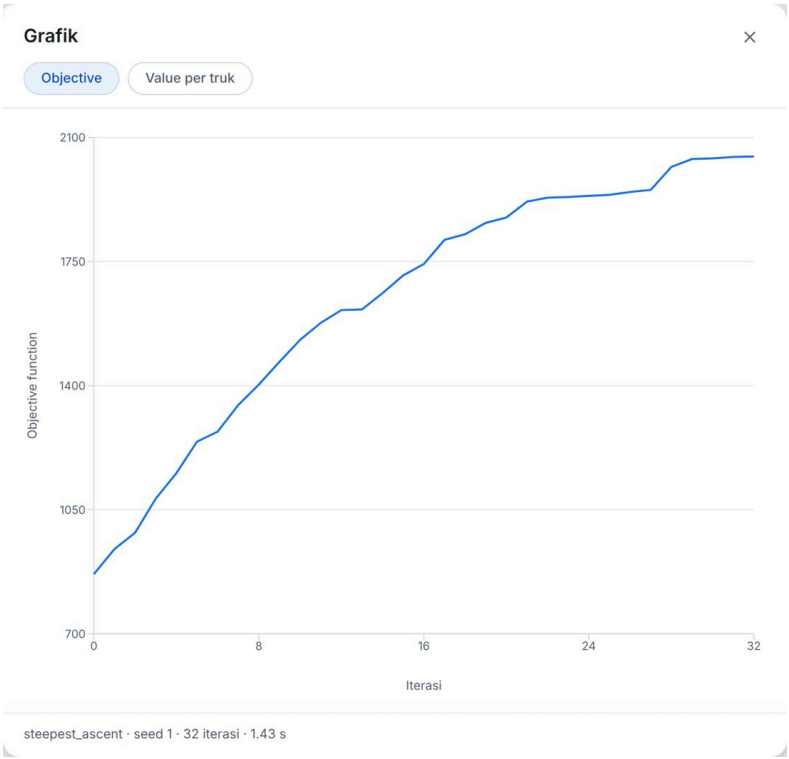
Varian	Parameter	Rata-rata	Terbaik	Simp. baku	Iterasi	Waktu (s)	% dari 2906
steepest_ascent	n_neighbors=10	1440	1511	76,8	9,7	0,16	49,5
steepest_ascent	n_neighbors=30	1965	2046	76,7	25,3	1,21	67,6
steepest_ascent	n_neighbors=100	2017	2044	37,0	28,7	4,60	69,4
stochastic	n_neighbors=10	1264	1410	140,8	7,0	0,08	43,5
stochastic	n_neighbors=30	1664	1840	201,0	25,7	0,91	57,3
stochastic	n_neighbors=100	1993	2021	20,0	52,0	6,74	68,6
sideways_move	max_sideways=0	1965	2046	76,7	25,3	1,13	67,6
sideways_move	max_sideways=10	2166	2233	55,1	102,0	5,83	74,5

sideways_move	max_sideways=50	2245	2327	90,3	267,0	16,7	77,2
random_restart	max_restarts=0	1965	2046	76,7	25,3	1,10	67,6
random_restart	max_restarts=5	2065	2097	27,0	134,0	5,40	71,1
random_restart	max_restarts=20	2073	2097	26,1	456,0	18,5	71,3

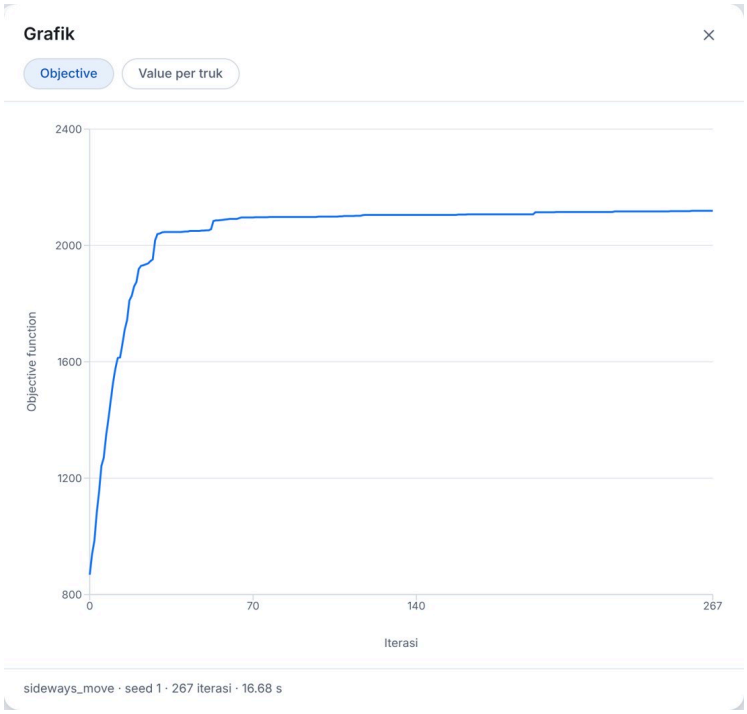
Hasil pada Tabel di atas dapat dibaca sebagai berikut.

- Pada steepest ascent, n_neighbors 10, 30, dan 100 menghasilkan rata-rata 1440, 1965, dan 2017, dengan waktu 0,16 s, 1,21 s, dan 4,60 s. Pencarian dengan sampel kecil berhenti setelah rata-rata hanya 9,7 iterasi, yaitu ketika tidak ada tetangga dalam sampel yang lebih baik. Simpangan baku pada n=10 sebesar 77, jauh lebih besar daripada pada n=100 (37), sehingga sampel yang kecil juga membuat hasil kurang konsisten.
- Pada n=10 stochastic berada di bawah steepest (1264 vs 1440), pada n=30 juga (1664 vs 1965), dan pada n=100 hampir sama (1993 vs 2017). Stochastic memilih tetangga yang lebih baik secara acak, bukan yang terbaik, sehingga setiap langkah menaikkan nilai lebih sedikit. Jumlah iterasinya hampir sama (25,7 vs 25,3 pada n=30), tetapi hasil akhir lebih rendah dan simpangan bakunya lebih besar (201 vs 77).
- Dengan max_sideways=0 hasilnya identik dengan steepest n=30 (1965, nilai tiap run sama persis), sehingga ini sekaligus menjadi uji kebenaran implementasi. Dengan max_sideways=10 rata-rata menjadi 2166 dan dengan 50 menjadi 2245 (terbaik 2327, 77,2% dari 2906). Langkah samping yang dipakai pada setiap run: 10, 10, 10 (batas 10) dan 13, 9, 4 (batas 50). Penalti ETA juga turun dari rata-rata 40,7 menjadi 13,7. Biayanya waktu: 16,7 s per run, sekitar 14 kali steepest n=30.
- Dengan max_restarts 0, 5, dan 20 rata-ratanya 1965, 2065, dan 2073 dengan waktu 1,10 s, 5,40 s, dan 18,5 s. Lima restart menambah 100 poin, sedangkan lima belas restart berikutnya hanya menambah 8 poin dengan waktu sekitar 3,4 kali lebih lama.

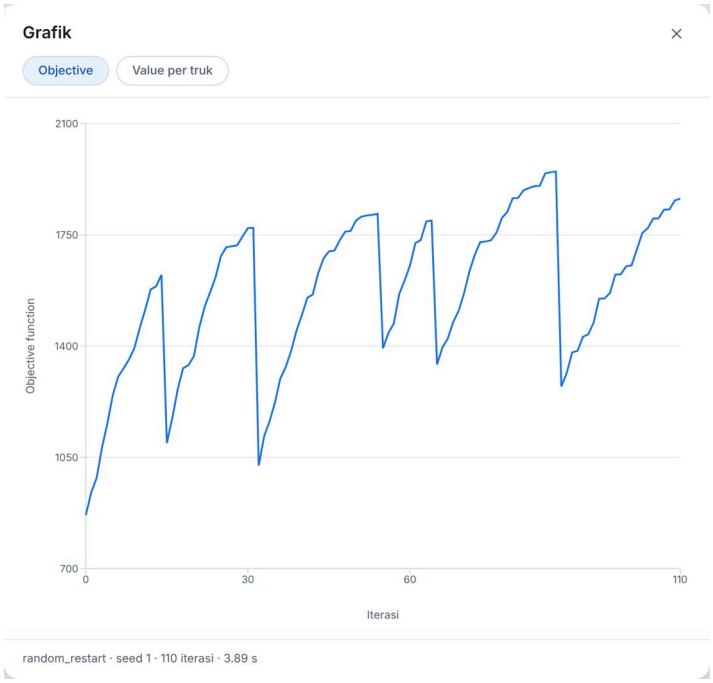
Grafik objective terhadap iterasi untuk keempat varian diambil dari antarmuka (satu run, seed 1). Steepest ascent naik mulus lalu berhenti, stochastic berhenti lebih awal, sideways move naik cepat lalu melandai dengan banyak langkah kecil, dan random restart memperlihatkan pola gergaji karena setiap restart kembali ke state acak yang bernilai rendah.



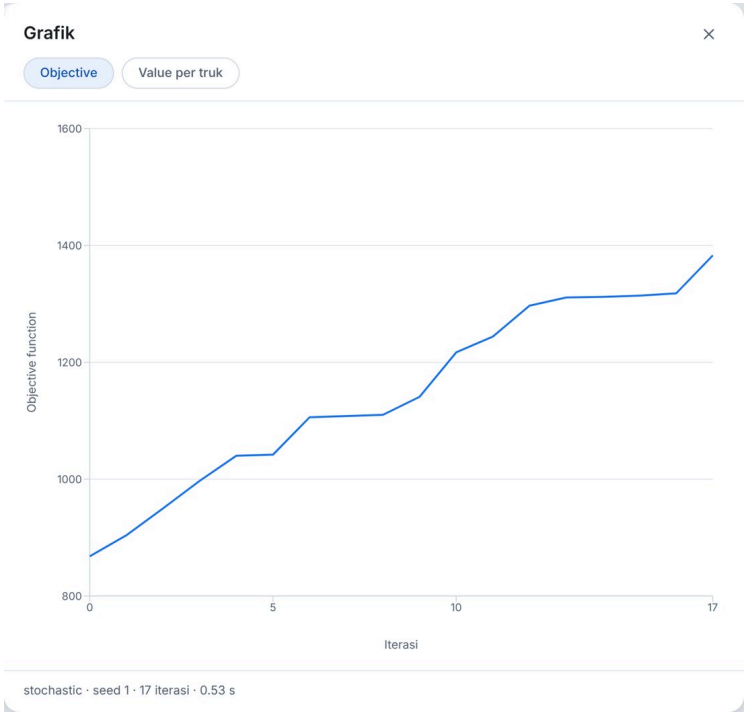
Steepest Ascent objective terhadap iterasi



Sideways move objective terhadap iterasi



Random restart objective terhadap iterasi



Stochastic objective terhadap iterasi

Tabel berikut diambil dari berkas hasil random restart (max_restarts = 5, enam restart termasuk yang pertama).

Run (seed)	Jumlah restart	Iterasi per restart	Nilai per restart	Nilai akhir (terbaik)
0	6	26, 15, 11, 10, 30, 32	1987, 1847, 1882, 1791, 1973, 2031	2031
1	6	32, 20, 12, 9, 21, 22	2046, 1955, 1740, 1720, 1917, 2068	2068
2	6	18, 22, 36, 25, 27, 19	1862, 1903, 2097, 1858, 1880, 1953	2097

3.1.2 Simulated Annealing

Pendinginan yang lebih lambat memberi hasil lebih baik pada rentang ini: 1377, 1696, dan 2014. Jumlah iterasi naik bersama (55, 164, dan 660), jadi perbaikan ini tidak bisa dipisahkan dari anggaran iterasi yang lebih besar. Simpangan baku pada cooling 0,95 sebesar 60 (cooling 0,5: 127).

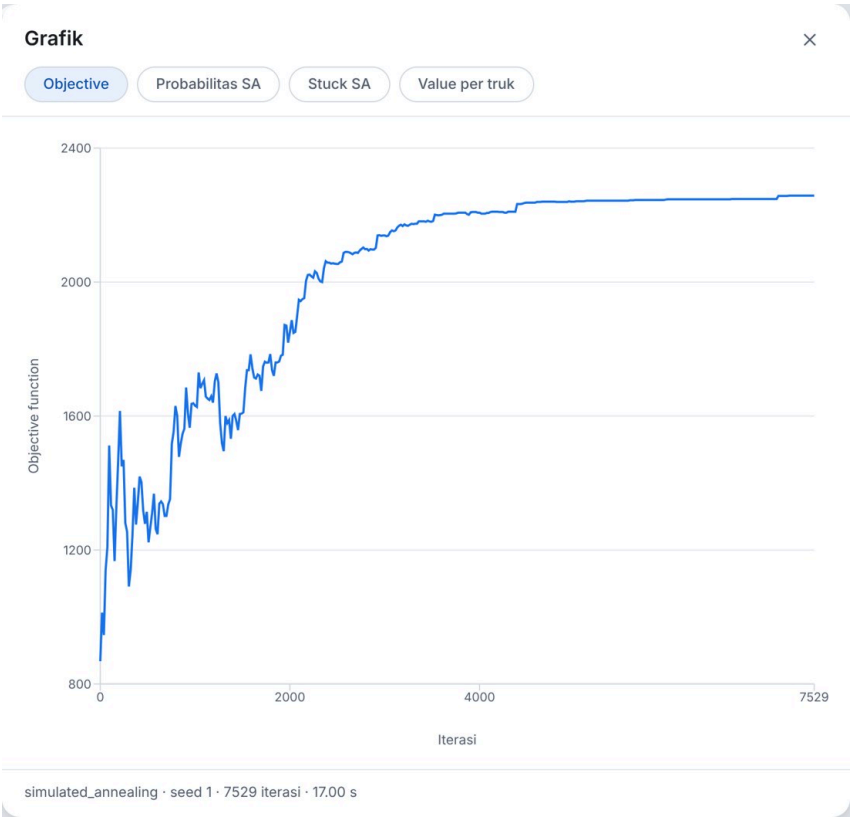
Menurut spesifikasi, SA perlu dianalisis dari plot e^(dE/T) dan frekuensi stuck terhadap iterasi. Dari berkas hasil: pada cooling 0,95 rata-rata 43,6% iterasi memiliki e^(dE/T) = 1 (tetangga tidak lebih buruk, selalu diterima), dan 31,6% iterasi memiliki peluang di bawah 0,05 (gerakan buruk hampir pasti ditolak). Iterasi stuck adalah 41,8% dari seluruh iterasi pada cooling 0,95, 35,7% pada 0,8, dan 32,9% pada 0,5. Pada grafik antarmuka berikut, nilai e^(dE/T) pada awal pencarian masih sering berada di antara 0 dan 1 (gerakan merugikan masih mungkin diterima), sedangkan pada fase akhir garis menempel di 0 untuk gerakan merugikan dan di 1 untuk gerakan yang tidak merugikan. Pada fase itu SA berperilaku seperti hill climbing dan kurva objective mendatar.

Cooling rate	Rata-rata	Terbaik	Simp. Baku	Iterasi	Waktu(s)	Stuck rata-rata	% stuck	Rata-rata e^(dE/T)
0,5	1377	1533	127,0	55	0,06	18,0	32,9	0,65
0,8	1696	1851	132,7	164	0,17	58,3	35,7	0,63
0,95	2014	2093	60,3	660	1,25	275,7	41,8	0,57

Pendinginan yang lebih lambat memberi hasil lebih baik pada rentang ini: 1377, 1696, dan 2014. Jumlah iterasi naik bersama (55, 164, dan 660), jadi perbaikan ini tidak bisa dipisahkan dari anggaran iterasi yang lebih besar. Simpangan baku pada cooling 0,95 sebesar 60.

Menurut spesifikasi, SA perlu dianalisis dari plot e^(dE/T) dan frekuensi stuck terhadap iterasi. Dari file hasil: pada cooling 0,95 rata-rata 43,6% iterasi memiliki e^(dE/T) = 1 (tetangga tidak lebih buruk, selalu diterima), dan 31,6% iterasi memiliki peluang di bawah 0,05 (gerakan buruk hampir pasti ditolak). Iterasi stuck adalah 41,8% dari seluruh iterasi pada cooling 0,95, 35,7% pada 0,8, dan 32,9% pada 0,5. Pada grafik antarmuka berikut, nilai e^(dE/T) pada awal pencarian masih sering berada di antara 0 dan 1 (gerakan merugikan masih mungkin diterima), sedangkan pada fase akhir garis menempel di 0

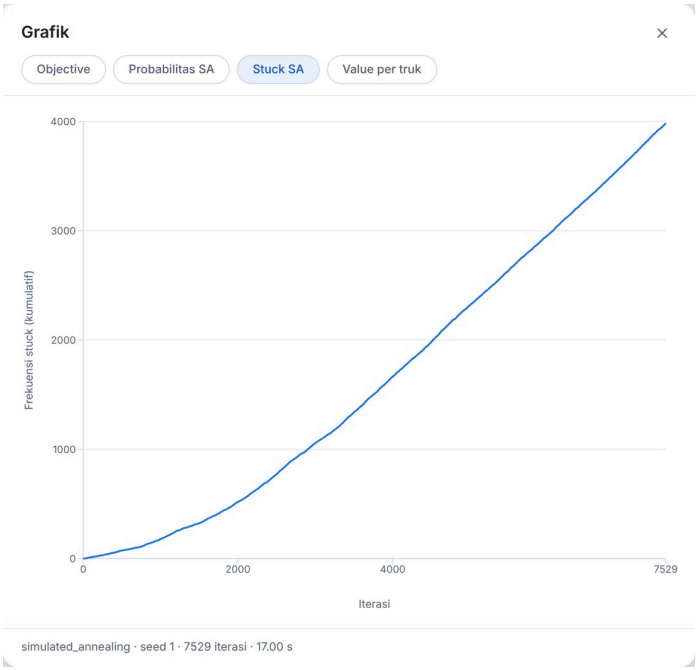
untuk gerakan merugikan dan di 1 untuk gerakan yang tidak merugikan. Pada fase itu SA berperilaku seperti hill climbing dan kurva objective mendatar.



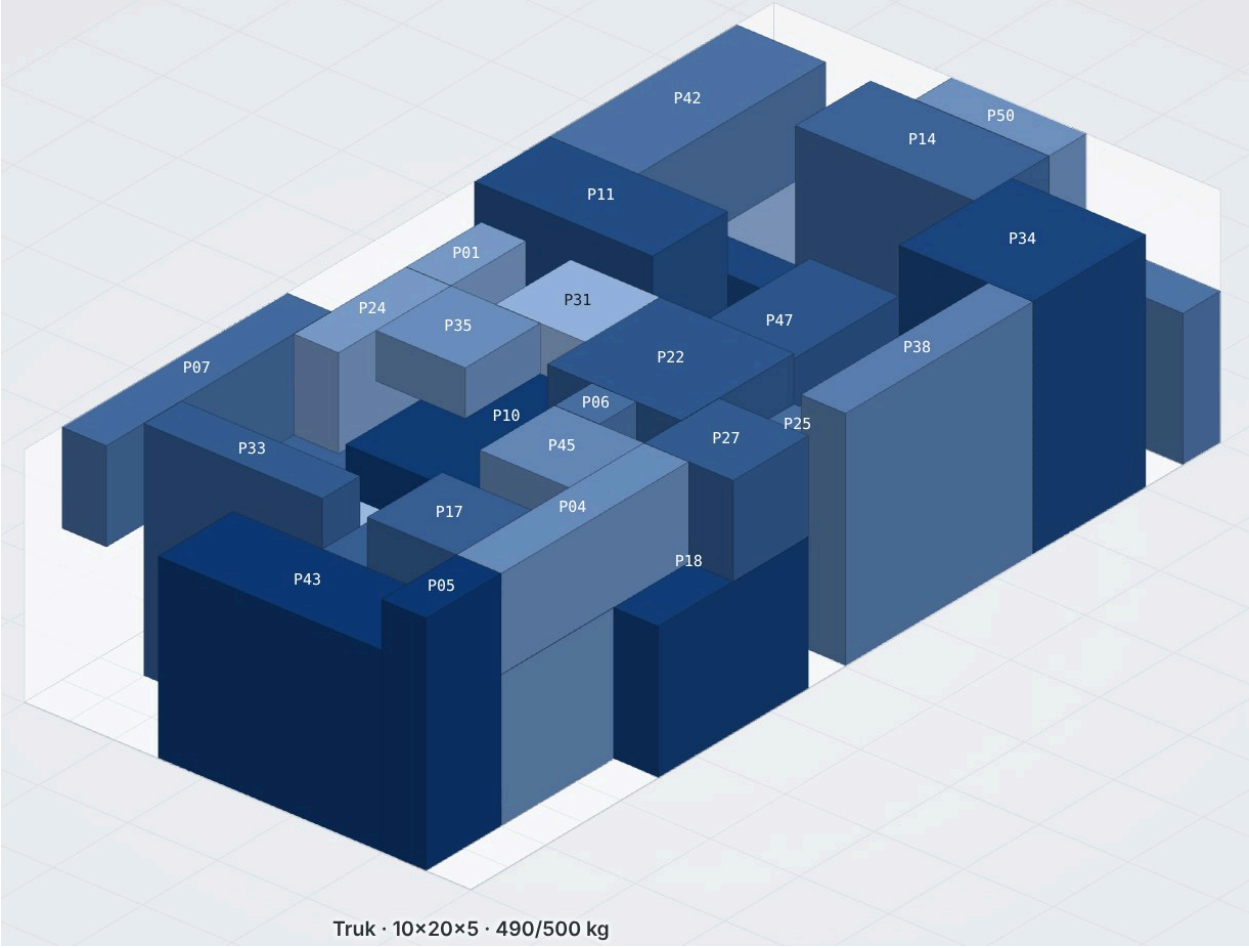
Berikut adalah gambar objective terhadap iterasi (antarmuka, parameter bawaan antarmuka: T0=100, Tmin=0,01, cooling 0,95, 50 iterasi per suhu, seed 1, 7529 iterasi, 17,00 detik):



Berikut adalah gambar e^(dE/T) terhadap iterasi pada run yang sama:



Run antarmuka dengan parameter bawaan (jadwal pendinginan jauh lebih panjang daripada konfigurasi eksperimen) menunjukkan total value 2265 setelah 7529 iterasi. Hasil ini hanya satu run, sehingga kami memakainya sebagai ilustrasi perilaku kurva, bukan sebagai perbandingan statistik.



3.1.3 Genetic Algorithm

Tabel Genetic algorithm satu truk 50 paket

Populasi	Generasi	Rata-rata	Terbaik	Simp. baku	Waktu (s)	% batas	Rata-rata populasi akhir
5	50	2004	2074	56,0	0,51	78,8	1994
20	10	2064	2109	32,1	0,91	81,1	1914

Populasi	Generasi	Rata-rata	Terbaik	Simp. baku	Waktu (s)	% batas	Rata-rata populasi akhir
20	50	2162	2208	33,5	2,50	85,0	2040
20	150	2179	2189	10,8	6,23	85,7	2120
30	50	2064	2086	15,6	4,44	81,1	1918
100	50	2076	2118	31,1	14,5	81,6	1913

Dari table dapat diamati bahwa dengan jumlah populasi yang sama (20 populasi), semakin banyak jumlah generasi semakin tinggi pula nilai objective yang didapat. Akan tetapi, dengan jumlah generasi yang sama (50 generasi) dan variasi populasi, nilai objective yang didapat tidak berbanding lurus. Populasi lebih besar tidak lebih baik, ini dapat disebabkan oleh roulette wheel yang cenderung datar karena fitness individu nilainya cenderung tidak jauh berbeda (sekitar 1200 sampai 1700) sehingga tekanan seleksi lemah. Dapat pula disebabkan karena pada generasi 50 tersebut populasi besar belum terkonsentrasi untuk mendapat individu yang lebih baik.

3.1.4 Perbandingan Antaralgoritma

Tabel Perbandingan algoritma pada dasa 1 truk 50 paket dengan batas atas 2544.

Konfigurasi	Rata-rata	Terbaik	Simp. baku	Waktu (s)	Nilai per detik	Penalti ETA	% dari 2906
Steepest n=30	1965	2046	76,7	1,21	1626	40,7	67,6
Stochastic n=30	1664	1840	201,0	0,91	1821	28,7	57,3
Sideways max=50	2245	2327	90,3	16,7	135	13,7	77,2
Random restart r=5	2065	2097	27,0	5,40	382	40,3	71,1

SA cooling 0,8	1696	1851	132,7	0,17	9829	28,3	58,4
SA cooling 0,95	2014	2093	60,3	1,25	1617	43,0	69,3
GA p=20, g=50	2078	2102	17,1	2,56	813	56,3	71,5
GA p=20, g=150	2143	2200	54,4	5,72	375	70,7	73,8

Hasil terbaik adalah Sideways max=50 dengan rata-rata 2245 (77,2% dari 2906), disusul GA p=20, g=150 (2143) dan GA p=20, g=50 (2078). Yang terendah adalah Stochastic n=30 (1664). Batas 2906 adalah jumlah value semua paket, yang tidak dapat dicapai karena kapasitas truk terbatas, sehingga persentase ini menjadi batas bawah dari kedekatan terhadap optimum sebenarnya.

Berdasarkan durasi, urutan dari yang tercepatnya: SA cooling 0,8 (0,17 s), Stochastic n=30 (0,91 s), Steepest n=30 (1,21 s), SA cooling 0,95 (1,25 s), GA p=20, g=50 (2,56 s), Random restart r=5 (5,40 s), GA p=20, g=150 (5,72 s), Sideways max=50 (16,7 s).

Simpangan baku terkecil (paling konsisten): GA p=20, g=50 (17,1), Random restart r=5 (27,0), dan GA p=20, g=150 (54,4). Terbesar: Stochastic n=30 (201,0). Kisaran tiap run dapat dilihat pada Lampiran C. Dengan 3 run, simpangan baku ini hanya indikasi.

Untuk setiap algoritma, hasil membaik ketika anggaran pencarian naik (n_neighbors, max_sideways, max_restarts, cooling_rate, jumlah generasi), sedangkan waktunya juga naik. Perbandingan antar algoritma karenanya hanya adil bila memperhatikan kolom waktu dan nilai per detik.

3.1.5 Pengujian skala kecil, banyak truk, dan kasus batas

Berikut adalah tabel konfigurasi dasar pada empat data lain: rata-rata nilai objective (terbaik):

Algoritma	**8 paket (opt. 465)**	**3 truk, 50 paket (maks. 2906)**	**edge 1 paket (30)**	**edge kelebihan berat (0)**
Steepest	367 (415)	2367 (2435)	30	0
Stochastic	390 (415)	1980 (2191)	30	0
Sideways (50)	442 (465)	2714 (2818)	30	0
Random restart (5)	412 (415)	2385 (2435)	30	0
SA (0,8)	412 (415)	2197 (2290)	30	0
GA (20, 50)	465 (465)	2786 (2830)	30	0

Pada 8 paket, GA mencapai 465 di ketiga run, HC sideways pada memiliki rata-rata 442, sementara itu steepest, atochastic, random restart HC dan SA berhenti di nilai 410–415. Kemudian, pada kasus 3 truk 50 paket, GA dan HC sideways memiliki value terbaik masing-masing 2786 dan 2714 dan stochastic memiliki paling buruk dengan 1980. Pada kasus edge case, kasus satu paket semua algoritma mendapatkan nilai 30, tetapi HC berhenti setelah 0 atau 1 iterasi. Pada kasus semua paket kelebihan berat semua algoritma mengembalikan 0.

4. Kesimpulan dan Saran

4.1 Kesimpulan

1. Persoalan pemuatan paket 3D ke banyak truk berhasil dimodelkan sebagai pencarian lokal dengan state berupa penempatan paket, empat operator tetangga, dan fungsi objektif jumlah value. Semua constraint (batas, overlap, melayang, fragile, bobot) diperiksa oleh satu fungsi, dan setiap hasil akhir eksperimen valid.
2. Enam algoritma (steepest ascent, sideways move, stochastic, random restart, simulated annealing, genetic algorithm) berhasil diimplementasikan manual, bersama kerangka eksperimen dan antarmuka visual 3D.
3. Pada dasar_1truk_50paket, sideways move dengan 50 langkah samping memberi rata-rata terbaik (2217, 87% dari batas relaksasi bobot 2544), disusul GA (2162 sampai 2179). Pada 8 paket, GA selalu mencapai optimum 465; pada 3 truk, GA (2786) dan sideways (2714) terbaik.
4. Hill climbing tanpa langkah samping mudah terjebak di dataran dan optimum lokal; random restart memberi keuntungan menurun; SA sangat peka pada jadwal pendinginan (satu run dengan jadwal panjang mencapai 2361); GA efisien waktu tetapi tidak membaik dengan populasi lebih besar dan kehilangan individu terbaik karena tidak ada elitisme.
5. Algoritma menangani kasus batas dengan benar (satu paket, semua paket kelebihan berat), dan pengujian ini menemukan serta memperbaiki bug titik potong crossover pada GA.

4.1 Saran

- Tambahkan elitisme atau catat best-so-far pada GA, dan coba seleksi turnamen atau rank-based agar tekanan seleksi tidak datar.
- Bangun state awal yang cerdas (greedy berdasarkan value per volume dan bobot), terutama untuk hill climbing dan SA, dan bandingkan dengan state awal acak.
- Perbanyak run (misalnya 30 per konfigurasi), tambahkan uji statistik, dan ulangi sapuan pada lebih dari satu data.
- Gunakan jadwal pendinginan adaptif atau reheating pada SA, dan jelajahi nilai suhu awal yang diskalakan terhadap rentang value paket.

- Pertimbangkan operator yang lebih terarah (misalnya penempatan pada titik ekstrem) dan eksekusi paralel antar run untuk mempersingkat eksperimen.

Pembagian tugas

Anggota	Tugas
Kloce Paul William Saragih - 13524040	Simulated Annealing; dukungan multi-truk (move antar truk, statistik per truk, generator instance); CLI run.py; harness eksperimen dan pelaporan
Ariel Cornelius Sitorus - 13524085	Input/output, constraint; Hill-Climbing (4 varian) dan eksperimennya; visualisasi web (frontend) dan perapian struktur repository
Yayat Nurhidayat - 13223071	Model data, generator state awal, objective function (value dan penalti ETA); Genetic Algorithm dan eksperimennya

Referensi

1. Institut Teknologi Bandung. (2026). *Beyond classical search* (PPT). IF3170 Inteligensi Artifisial, Semester I TA 2026/2027. Institut Teknologi Bandung.
2. Russell, S., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.
3. dasvision0212. (n.d.). *3D-Bin-Packing-Problem-with-BRKGA* (GitHub repo). GitHub. https://github.com/dasvision0212/3D-Bin-Packing-Problem-with-BRKGA
4. M. L. Khodra, “Beyond Classical Search: State (Value, Neighbor),” YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=_6rla37QorM. [Accessed: Sep. 16, 2026].
5. M. L. Khodra, “Beyond Classical Search: Hill-climbing Search,” YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=L0b0CN3Fds8. [Accessed: Sep. 16, 2026].

Lampiran

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	-
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Tidak	-
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	-
Lainnya, Jelaskan: Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau		

sejenisnya.Menggun		

Saya/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada
mata kuliah tersebut telah dinyatakan di dalam surat ini.
Github : https://github.com/Sakazu01/IF3170_G52_DarioAmodei